

미니프로젝트2(MMDetection+voc2yolo) 내용 정리

```
1 import torch, mmdet
2 from mmdet import Config
3 from mmdet.apis import init_detector, inference_detector, set_random_seed, train_detector
4 from mmdet.datasets.builder import DATASETS
5 from mmdet.datasets.coco import CocoDataset
6 from mmdet.datasets import build_dataset
7 from mmdet.models import build_detector
8 from mmdet.runner import get_dist_info, init_dist
9 from mmdet.utils import collect_env, get_root_logger, setup_multi_processes
```

필요한 라이브러리 불러오기

```
"""Dataset Register"""
@DATASETS.register_module(force=True) # 강제로 클래스 정보 수정할 거기 때문에 넣음
class WineLabelsDataset(CocoDataset): # 상속받는 건 CocoDataset | json 파일에 적힌 label(=class 정보)대로 입력하면 됨
    # mmdet\datasets\coco.py
    CLASSES = ('wine-labels', 'AlcoholPercentage', 'Appellation AOC DOC AVARegion', 'Appellation QualityLevel', 'CountryCountry')
```

데이터셋 등록

Force=True로 강제로 클래스 정보 수정

```
"""Config file"""
# cascade_rcnn(겁나 무거움), dynamic_rcnn, fast_rcnn, mask_rcnn(segmentation 할 때 씬) ! 애네를 좀 많이 씬
config_file = './configs/dynamic_rcnn/dynamic_rcnn_r50_fpn_1x_coco.py' # r50은 resnet50을 의미함
cfg = Config.fromfile(config_file) # 파일 읽어들이기
# print(cfg.pretty_text) # 해당 config file의 아키텍처 구조나 정보를 보고 싶을 때 사용
```

Config 파일 불러와서 다루기

```
"""Learning Rate setting"""
# single GPU default --> 0.0025
cfg.optimizer.lr = 0.0025 # 0.02 / 8 : 1개의 GPU로 하는 거라서 8로 나눠줌. 원래 학습을 8개 GPU로 하기 때문

"""Dataset setting"""
cfg.dataset_type = 'WineLabelsDataset' # configs\_base\_datasets\coco_detection.py 참고
cfg.data_root = './dataset'
```

Learning rate랑 dataset 설정(gpu 개수에 따른 설정이 좀 달라짐, 데이터셋은 위에서 선언한 거 그대로 사용)

```
"""Train, Val, Test dataset의 type, data root, annotation file, img_prefix 설정"""
cfg.data.train.type = 'WineLabelsDataset' # train
cfg.data.train.ann_file = './dataset/train/_annotations.coco.json' # 이 파일을 읽고 처리해줘!
cfg.data.train.img_prefix = './dataset/train/' # 뒤에 알아서 train 붙을 거기 때문에 여기까지만 써주면 됨
cfg.data.val.type = 'WineLabelsDataset' # valid
cfg.data.val.ann_file = './dataset/valid/_annotations.coco.json'
cfg.data.val.img_prefix = './dataset/valid/'
cfg.data.test.type = 'WineLabelsDataset' # test
cfg.data.test.ann_file = './dataset/test/_annotations.coco.json'
cfg.data.test.img_prefix = './dataset/test/'
```

모델 학습에 사용할 데이터셋 경로 지정

```

cfg.model.roi_head.bbox_head.num_classes = 13 # class number 수정하기 | models쪽 타고 넘어감
cfg.model.rpn_head.anchor_generator.scales = [4] # 작은 객체를 잡기 위해 anchor size를 변경할 것임 | default=8 --> 4
cfg.load_from = './dynamic_rcnn_r50_fpn_1x-62a3f276.pth' # pretrained-model pt파일 불러오기, 다운로드 받아서 넣어야 함
cfg.work_dir = './work_dirs/0130' # 학습한 모델 저장 경로 지정

cfg.lr_config.warmup = None # learning rate hyperparameter 설정
cfg.log_config.interval = 10 # 10번째마다 보여줘

```

모델 base 타고 들어가서 뭐 바뀌야 하는지 확인하고 데이터셋에 맞게 갯수 수정, 경로 지정 등

```

'''[평가]COCO dataset evaluation type = bbox 지정''' # mAP로 놓는 경우도 있음
# mAP IoU threshold 0.5 ~ 0.95 변경하면서 측정
cfg.evaluation.metric = 'bbox'
cfg.evaluation.interval = 10 # 평가 몇 번에 한 번씩 할래? 보통 10번 정도마다 봄
cfg.checkpoint_config.interval = 10 # 모델 파일 중간 저장 몇 번에 한 번씩 할래?

cfg.runner.max_epochs = 88 # epoch 설정 | 기본값 : 8(GPU 갯수) x 12 = 96, 지금은 GPU 1개라 저렇게 설정
cfg.seed = 777 # 고정된 seed값 지정
cfg.data.samples_per_gpu = 6 # single GPU에서 통용되는 값 | batch_size라고 인식하면 됨
cfg.data.workers_per_gpu = 2 # num_workers, single GPU에서 통용되는 값 | 애네는 거의 고정값
# print('cfg.data >>', cfg.data)

```

평가 및 중간 저장 간격 설정, 단일 gpu로 통상적으로 사용되는 값 설정

```

cfg.gpu_ids = range(1) # GPU 사용 갯수
cfg.device = 'cuda' # device 지정
set_random_seed(777, deterministic=False) # 고정된 seed값 지정
# print('cfg info show >>> ', cfg.pretty_text)

datasets = [build_dataset(cfg.data.train)] # train용 데이터를 생성하고 올리자
# print('datasets[0]', datasets[0])

datasets[0].__dict__.keys() # datasets[0].__dict__ variables key val

model = build_detector(cfg.model, train_cfg=cfg.get('train_cfg'), test_cfg=cfg.get('test_cfg'))
model.CLASSES = datasets[0].CLASSES
# print(model.CLASSES)

if __name__ == '__main__': # 애는 num_worker로 인한 이유인지 이런 방식으로 실행해야 함. ipynb에서 할거면 조건문 없이 실행
    train_detector(model, datasets, cfg, distributed=False, validate=True)

```

연산 진행할 device 설정, 데이터셋과 모델 빌드하고 num_worker로 인한 조건문으로 통해 실행문 작성

```

Evaluate annotation type *bbox*
DONE (t=2.65s).
Accumulating evaluation results...
DONE (t=1.04s).
2023-01-30 23:05:30.050 - mmdet - INFO -
Average Precision (AP) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.294
Average Precision (AP) @[ IoU=0.50 | area= all | maxDets=1000 ] = 0.436
Average Precision (AP) @[ IoU=0.75 | area= all | maxDets=1000 ] = 0.341
Average Precision (AP) @[ IoU=0.50:0.95 | area= small | maxDets=1000 ] = 0.174
Average Precision (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=1000 ] = 0.282
Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=1000 ] = 0.228
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.476
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=300 ] = 0.476
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=1000 ] = 0.476
Average Recall (AR) @[ IoU=0.50:0.95 | area= small | maxDets=1000 ] = 0.304
Average Recall (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=1000 ] = 0.483
Average Recall (AR) @[ IoU=0.50:0.95 | area= large | maxDets=1000 ] = 0.434

```

80epoch에서 평가 결과는 위와 같이 나왔다

[선생님께 질문 드렸던 내용 정리]

Q. MMDetection도 사정상 중도에 중지 후 yolo처럼 이어서 실험이 가능한가요?

A. 가능함. “./tool/train.py” 쪽에 인자 옵션 줄 수 있음

```

23
24
25 def parse_args():
26     parser = argparse.ArgumentParser(description='Train a detector')
27     parser.add_argument('config', help='train config file path')
28     parser.add_argument('--work-dir', help='the dir to save logs and models')
29     parser.add_argument(
30         '--resume-from', help='the checkpoint file to resume from')
31     parser.add_argument(
32         '--auto-resume',
33         action='store_true',
34         help='resume from the latest checkpoint automatically')
35     parser.add_argument(
36         '--no-validate',
37         action='store_true',
38         help='whether not to evaluate the checkpoint during training')
39     group_gpus = parser.add_mutually_exclusive_group()
40     group_gpus.add_argument(
41         '--gpus',
42         type=int,

```

Q. 말씀하신 것처럼 다른 모델로 돌려보려면 yolo처럼 이전 실험의 부산물인 데이터셋에 있는 cache를 지워줘야 하나요? 아직 돌려보지 못해서 cache 생성 여부는 모르는 상태입니다.

A. 따로 생성되지 않기 때문에 지워줄 필요 없음

Q. 천천히 하나씩 print문 찍어보면서 돌려보려고 리뷰해주신 내용을 주피터노트북 상에서 돌려보고 싶은데 이럴 땐 최하단에 실행문으로 작성한 if __name__ == '__main__' 으로 실행하지 않고 그냥 바로 train_detector로 입력해서 해당 셀을 실행해보면 될까요?

A. 맞음. 근데 기존 .py 스크립트로 88epoch 돌렸더니 14시 20분부터 돌렸는데 23시 54분이 되어서야 88epoch 가 다 돌아서 ipynb로 돌릴 수가 없었음

```

2023-01-30 23:53:37.313 - mmdet - INFO - Epoch [88][490/529] lr: 2.500e-05, eta: 0:00:26, time: 0.862, data_time: 0.026, memory: 13291, loss_rpn_cls: 0.0046, loss_rpn_bbox: 0.0192, loss_cls: 0.1713, acc: 94.2773, loss_bbox: 0.1026, loss: 0.2978
2023-01-30 23:53:44.193 - mmdet - INFO - Epoch [88][500/529] lr: 2.500e-05, eta: 0:00:19, time: 0.886, data_time: 0.024, memory: 13291, loss_rpn_cls: 0.0042, loss_rpn_bbox: 0.0143, loss_cls: 0.1510, acc: 94.8470, loss_bbox: 0.0876, loss: 0.2570
2023-01-30 23:53:50.968 - mmdet - INFO - Epoch [88][510/529] lr: 2.500e-05, eta: 0:00:12, time: 0.678, data_time: 0.027, memory: 13291, loss_rpn_cls: 0.0044, loss_rpn_bbox: 0.0160, loss_cls: 0.1483, acc: 94.9154, loss_bbox: 0.0827, loss: 0.2514
2023-01-30 23:53:57.523 - mmdet - INFO - Epoch [88][520/529] lr: 2.500e-05, eta: 0:00:06, time: 0.658, data_time: 0.025, memory: 13291, loss_rpn_cls: 0.0045, loss_rpn_bbox: 0.0157, loss_cls: 0.1501, acc: 94.7721, loss_bbox: 0.0876, loss: 0.2578
2023-01-30 23:54:04.111 - mmdet - INFO - Saving checkpoint at 88 epochs

(mmdetection) C:\Users\labadmin\Desktop\mmdetection>

```


VOC data to YOLO

```

1  import os, glob, cv2
2  from xml.etree.ElementTree import parse
3
4
5  xml_dir = "./wine_labels_voc_dataset"
6  class voc_to_yolo_converter() :
7      def __init__(self, xml_paths) : # xml 파일 경로를 받아와야 하니까 인자로 넣자
8          self.xml_path_lich = glob.glob(os.path.join(xml_paths, "*.xml"))
9          # print(self.xml_path_lich)
10
11      def voc_xyxy_show(self, x1, y1, x2, y2, label_name, file_name) : # ?????
12          image_path = os.path.join("./wine_labels_voc_dataset", file_name)
13          image = cv2.imread(image_path)
14          img_rect = cv2.rectangle(image, (x1, y1), (x2, y2), (0, 255, 0), 2)
15          # cv2.imshow("test", image)
16          # cv2.waitKey(1)
17          return img_rect

```

필요한 라이브러리 호출, 데이터 경로 지정, 초기 설정으로 파일 모두 스캔됨

```

19  def get_voc_to_yolo(self) :
20      for xml_path in self.xml_path_lich :
21          # print(xml_path)
22          tree = parse(xml_path) # 불러와서 읽고 parse를 하자
23          root = tree.getroot() # <annotations> | 이 밑은 root.findall('root')로 해서 접근하면 된다
24
25          file_name = root.find('filename')
26
27          size_meta = root.findall('size') # get image size
28          img_width = int(size_meta[0].find('width').text)
29          img_height = int(size_meta[0].find('height').text)
30          # print(img_width, img_height)
31
32          object metas = root.findall('object') # get object meta
33          for object_meta in object metas : # get box info
34              object_label = object_meta.find('name').text # label name | <object><name>
35
36              """bounding box 값을 찾아주자"""
37              xmin = object_meta.find('bndbox').findtext('xmin')
38              xmax = object_meta.find('bndbox').findtext('xmax')
39              ymin = object_meta.find('bndbox').findtext('ymin')
40              ymax = object_meta.find('bndbox').findtext('ymax')
41              # print(object_label, xmin, xmax, ymin, ymax)

```

함수 설정하고 파싱할 tree, root 설정해주고 필요로 하는 정보 검색해서 찾게 함. 애네는 데이터 내부가 모두 문자열이라 int써서 숫자로 바꿔줘야 함

```

44
45     """voc to yolo 변환 공식 = normalization"""
46     yolo_x = round(((int(xmin) + int(xmax)) / 2) / img_width, 6)
47     yolo_y = round(((int(ymin) + int(ymax)) / 2) / img_height, 6)
48     yolo_w = round((int(xmax) - int(xmin)) / img_width)
49     yolo_h = round((int(ymax) - int(ymin)) / img_height)
50     # print(yolo_x, yolo_y, yolo_w, yolo_h)
51
52
53     """디버깅용"""
54     if __name__ == "__main__" :
55         test = voc_to_yolo_converter(xml_dir)
56         test.get_voc_to_yolo()

```

voc로부터 뽑은 값을 yolo 형식으로 전환하도록 공식 작성, 잘 나오나 보기 위해 조건문 걸고 실행